

Documentación de Formalismos

Proyecto Final de PLN

Edgar Andres Vargas Garcia, Juan Pablo Escobar Viveros
Jhojan Stiven Castaño Jejen
Ingeniería de Sistemas
Universidad del Valle
Semestre 2026–1

I. DESCRIPCIÓN DEL SISTEMA

El proyecto desarrollado consiste en un juego de texto tipo *escape room* con ambientación RPG en español. El usuario interactúa mediante comandos escritos en lenguaje natural, como *tomar la llave*, *abrir el cofre con la llave*, *leer el pergamino* o *ir al norte*. El sistema analiza cada entrada, extrae su intención y actualiza el estado del mundo del juego.

La arquitectura general implementada es:

texto $\rightarrow$ DFA $\rightarrow$ CFG
 $\rightarrow$ parser $\rightarrow$ árbol
 $\rightarrow$ DCG $\rightarrow$ ATN

Con esta cadena se integran las herramientas formales vistas en el curso: autómata finito, gramática de contexto libre, parser, árboles de derivación, manejo de ambigüedad, DCG/unificación y ATN.

II. AUTÓMATA FINITO PARA TOKENIZACIÓN

El primer componente es un autómata finito determinista usado para separar la entrada en palabras. Formalmente:

$$M = (Q, \Sigma, \delta, q_0, F)$$

donde:

$$Q = \{Q_0, Q_1\}, \quad q_0 = Q_0, \quad F = \{Q_1\}$$

$$\Sigma = \text{letras del español} \cup \text{separadores}$$

El estado Q_0 representa la lectura fuera de una palabra y Q_1 la lectura dentro de una palabra. La función de transición es:

$$\delta(Q_0, \text{letra}) = Q_1, \quad \delta(Q_0, \text{sep}) = Q_0$$

$$\delta(Q_1, \text{letra}) = Q_1, \quad \delta(Q_1, \text{sep}) = Q_0$$

Cuando se pasa de Q_1 a Q_0 , se emite el lexema acumulado. Luego, el lexema se normaliza y se clasifica como VERBO, ARTICULO, SUSTANTIVO, PREPOSICION, ADJETIVO, DIRECCION o DESCONOCIDO.

III. GRAMÁTICA DE CONTEXTO LIBRE

La gramática principal se define como:

$$G = (V, T, P, S)$$

donde:

$$V = \{S, \text{Comando}, FV, FN, FP\}$$

$$T = \{VERBO, ARTICULO, SUSTANTIVO, PREPOSICION, ADJETIVO, DIRECCION\}$$

$$S = S$$

Las producciones principales son:

$$S \rightarrow \text{Comando}$$

$$\text{Comando} \rightarrow FV \mid FV FN \mid FV FN FP \mid FV FP$$

$$FV \rightarrow VERBO$$

$$FN \rightarrow ARTICULO SUSTANTIVO$$

$$FN \rightarrow ARTICULO ADJETIVO SUSTANTIVO$$

$$FN \rightarrow ARTICULO SUSTANTIVO ADJETIVO$$

$$FN \rightarrow SUSTANTIVO \mid ADJETIVO SUSTANTIVO \mid SUSTANTIVO ADJETIVO$$

$$FN \rightarrow FN FP$$

$$FP \rightarrow PREPOSICION FN \mid PREPOSICION DIRECCION$$

Esta CFG permite reconocer comandos simples y compuestos. Por ejemplo, acepta *inventario*, *tomar la llave*, *ir al norte*

y *abrir el cofre con la llave*. La producción $FN \rightarrow FN FP$ permite representar adjunción de sintagmas preposicionales y genera posibles ambigüedades.

IV. PARSER Y ÁRBOLES DE DERIVACIÓN

Después de obtener los tokens, el sistema usa un parser tipo Earley para revisar si el comando escrito por el jugador tiene una estructura válida según la gramática definida. Este parser aparece implementado en el proyecto y permite trabajar directamente con las reglas de la gramática sin tener que simplificarlas demasiado. Por ejemplo, un comando como:

Comando $\rightarrow$ *FV FN FP*

El parser usa ítems de la forma:

$[A \rightarrow \alpha \bullet \beta, i]$

y construye un *chart* con tres operaciones: *Predict*, *Scan* y *Complete*. Si la entrada cumple la gramática, se genera uno o más árboles de derivación. Por ejemplo:

tomar la llave

se analiza como:

$S \Rightarrow \text{Comando} \Rightarrow FV FN$

$FV \Rightarrow \text{VERBO}$, $FN \Rightarrow \text{ARTICULO SUSTANTIVO}$

Si la entrada no cumple la gramática, por ejemplo *la llave*, el sistema no genera acción y responde con un mensaje de error controlado.

V. AMBIGÜEDAD Y SEMÁNTICA

El manejo de ambigüedad aparece cuando una entrada genera más de un árbol de derivación. Un caso es:

examinar el cofre con la llave

Este comando puede entenderse de dos maneras. La primera es que el jugador quiere examinar el cofre usando la llave. La segunda es que el jugador se refiere a un cofre relacionado con la llave. Esta situación se presenta porque la gramática permite que una frase como *con la llave* acompañe al objeto principal del comando.

Luego, la capa semántica convierte el análisis del comando en una intención más clara para el juego, usando una estructura como:

$\text{accion}(\text{verbo}, \text{objeto}, \text{complemento})$

Ejemplos:

$\text{tomar la llave} \Rightarrow \text{accion}(\text{tomar}, \text{objeto}(\text{llave}), \text{ubicacion}(\text{None}))$

$\text{ir al norte} \Rightarrow \text{accion}(\text{ir}, \text{None}, \text{direccion}(\text{norte}))$

$\text{abrir el cofre con la llave} \Rightarrow \text{accion}(\text{abrir}, \text{objeto}(\text{cofre}), \text{instrumento}(\text{objeto}(\text{llave})))$

Las preposiciones reciben roles semánticos: *con* se interpreta como instrumento, *en* como ubicación, *al* como destino y *desde* como origen.

VI. ATN Y ESTADO DEL JUEGO

La ATN modela el mundo del juego y decide si una acción es válida según el estado actual. Sus estados son las salas: celda, pasillo, biblioteca y salida. Las transiciones conectan estas salas mediante direcciones como norte, sur, este y oeste. Además, se usan registros aumentados: inventario, objetos por sala y banderas lógicas.

Algunas banderas son:

puerta_celda_abierta, *cofre_abierto*,
pergamino_leido

Por ejemplo, al inicio el jugador está en la celda y no puede ir al norte porque la puerta está cerrada. Si ejecuta *tomar la llave* y luego *abrir la puerta*, se activa la bandera *puerta_celda_abierta* y se desbloquea la transición al pasillo. En la biblioteca, *abrir el cofre con la llave* hace aparecer el pergamino, y *leer el pergamino* desbloquea la salida.

VII. CASOS PARA LA DEMO

Para la demostración se usarán entradas como: *tomar la llave*, *abrir la puerta*, *ir al norte*, *examinar el cofre con la llave*, *abrir el cofre con la llave*, *leer el pergamino* y *la llave*. Estas pruebas muestran el funcionamiento del lexer, la CFG, el parser, la semántica, la ATN, el manejo de ambigüedad y el manejo de entradas inválidas.

VIII. REPOSITORIO

Repositorio del proyecto:

https://github.com/JuanPE-PNG/PLN_Proyecto

El repositorio contiene el código fuente, el archivo README.md, las instrucciones de ejecución y los commits del desarrollo.

REFERENCIAS

- [1] J. Earley, "An efficient context-free parsing algorithm," *Communications of the ACM*, vol. 13, no. 2, pp. 94–102, 1970. Disponible en: <https://disi.unitn.it/~bernardi/Courses/CompLing/Papers/earley70.pdf>
- [2] F. C. N. Pereira and D. H. D. Warren, "Definite clause grammars for language analysis—A survey of the formalism and a comparison with augmented transition networks," *Artificial Intelligence*, vol. 13, no. 3, pp. 231–278, 1980.